数据结构核心考点期末复习讲义（Modern C++ 重构版）

> **修订声明**：本文档基于原教学大纲全面重构。修正了原稿中公式丢失、断句错乱及古早语法问题；核心代码全部剥离 C 风格内存管理，规范化为现代 C++ 面向对象与引用传参写法。

导读：期末答题避坑指南（Akiro 批注）

1. **语言标准的撕裂**：国内期末卷面答题，**手写代码一律用裸指针和引用（`&`）**，千万不要在卷面上写 `std::shared_ptr` 或 `auto`，老教授大概率判错。但动态内存分配必须用 `new / delete`，剔除 `malloc / free`。
2. **下标陷阱（卷面生死线）**：严蔚敏教材中，顺序表、字符串、数组的有效元素默认**从下标 1 开始**（`s[0]` 往往用来存长度或哨兵）。**但现代 C++ 规范是从 0 开始。**在本讲义中，**所有代码均严格采用 0-based 标准**，但在核心公式处我为你标注了“教材版面规约”，进考场前务必和你们授课老师的 PPT 规约对齐！

第 1 章 绪论

1.1 数据结构研究内容

针对**非数值计算**的程序设计问题，研究计算机的操作对象以及它们之间的关系和操作。

Plaintext

```
数据结构涵盖内容：
  ├─ 逻辑结构
  │     ├─ 线性结构（线性表、栈、队列、串、数组）
  │     └─ 非线性结构（树结构、图结构）
  ├─ 物理（存储）结构
  │     ├─ 顺序存储结构
  │     ├─ 链式存储结构
  │     ├─ 索引存储结构
  │     └─ 散列（哈希）存储结构
  └─ 数据运算：插入、删除、修改、查找、排序
```

1.2 基本概念

- **数据 (Data)**：所有能被计算机识别、存储和处理的符号的集合。
- **数据元素 (Data Element)**：数据的**基本单位**，具有完整确定的实际意义。
- **数据对象 (Data Object)**：具有相同性质的数据元素的集合，是数据的一个子集。
- **数据结构 (Data Structure)**：相互之间存在一种或多种特定关系的数据元素的集合，数学模型表示为：

  $$Data_Structure = (D, R)$$

  (其中 D 为数据元素的有限集，R 为 D 上关系的有限集)

- **数据类型 (Data Type)**：一个**值的集合**和定义在该值上的**一组操作**的总称。
- **抽象数据类型 (ADT, Abstract Data Type)**：由用户定义的一个数学模型与定义在该模型上的一组操作，它由基本的数据类型构成。

1.3 算法及分析

- **算法的五个基本特征**：输入、输出、**有穷性**、**确定性**、**可行性**。
- **算法设计四大要求**：① 正确性、② 可读性、③ 健壮性、④ 高效率与低存储量需求。
- **算法效率评估指标**：
  - **时间复杂度**：利用执行次数的频度函数 $T(n) = O(f(n))$ 来估算。
  - **空间复杂度**：算法运行过程中占用辅助空间的大小 $S(n) = O(g(n))$。
  - **稳定性**：*(点出概念，第9章排序重点考)*。

第 2 章 线性表（含数组与稀疏矩阵）

2.1 线性表逻辑与物理结构

逻辑定义：由 n ($n \ge 0$) 个数据元素组成的有限序列 $(a_1, a_2, \dots, a_n)$。$n=0$ 时称为**空表**。

存储结构	核心物理特点	空间密度	存取方式	适用场景
顺序表	逻辑相邻，**物理内存绝对相邻**	1.0 (极高)	**随机存取** $O(1)$	查多改少，表长固定
链式表	逻辑相邻，物理内存随意分布	<1.0	**顺序存取** $O(n)$	频繁增删，表长动态

2.2 顺序表核心运算效率

- **修改**：核心语句 `v[i] = x;`，时间复杂度 **$O(1)$**。
- **插入**：在第 i 个位置前插入，平均移动次数为 $\frac{n}{2}$，时间复杂度 **$O(n)$**。
  (合法位置条件：$1 \le i \le n+1$)
- **删除**：删除第 i 个元素，平均移动次数为 $\frac{n-1}{2}$，时间复杂度 **$O(n)$**。

2.3 单链表（Modern C++ 代码实战）

考题 1：生成由 26 个英文字母组成的单链表

C++

```
#include <iostream>

struct ListNode {
    char data;
    ListNode* next;
    // C++ 构造函数初始化列表，卷面加分项
    ListNode(char val = 0, ListNode* p = nullptr) : data(val), next(p) {}
};
```

```
// 规范写法：带头结点的尾插法构建
ListNode* buildAlphaList() {
    ListNode* head = new ListNode(); // 头结点（不存有效字符）
    ListNode* tail = head;

    for (int i = 0; i < 26; ++i) {
        tail->next = new ListNode('a' + i);
        tail = tail->next;
    }
    return head;
}

void printList(ListNode* head) {
    ListNode* p = head->next; // 滤过头结点
    while (p != nullptr) {
        std::cout << p->data << " ";
        p = p->next;
    }
    std::cout << std::endl;
}
```

考题 2：读取单链表第 i 个元素

C++

```
ListNode* getElem(ListNode* head, int i) {
    if (i < 1) return nullptr;
    ListNode* p = head->next;
    int j = 1;

    while (p != nullptr && j < i) {
        p = p->next;
        ++j;
    }
    return p; // 若 i 超过表长，自然返回 nullptr
}
```

考题 3：单链表的插入与删除核心原子逻辑

> ⚠️ **卷面黄金铁律**：链表插入动作，**先接后链**；删除动作，**先接后销**。顺序严禁颠倒！

- **单链表后插（在结点 `p` 后插入新结点 `s`）**：

  C++

  ```
  s->next = p->next;
  p->next = s;
  ```

- **单链表删除（已知前驱结点 `p`，删除其后继 `q`）**：

  C++

```
ListNode* q = p->next;
p->next = q->next;
delete q; // C++ 释放内存用 delete
```

考题 4：双向链表 (Doubly Linked List) 核心逻辑

设指针 `p` 指向链表第 i 个元素，要在 `p` **之前**插入新结点 `s`：

C++

```
s->prior = p->prior;
p->prior->next = s;
s->next = p;
p->prior = s;
```

2.4 数组的内存寻址公式（必考计算题）

设二维数组 $A[c_1 \dots d_1][c_2 \dots d_2]$，基址为 $LOC(a_{c_1, c_2})$，每个元素占 L 个字节：

1. **行优先存储地址公式**：

   $$LOC(a_{i,j}) = LOC(a_{c_1, c_2}) + \big[(i - c_1) \times (d_2 - c_2 + 1) + (j - c_2) \big] \times L$$

2. **列优先存储地址公式**：

   $$LOC(a_{i,j}) = LOC(a_{c_1, c_2}) + \big[(j - c_2) \times (d_1 - c_1 + 1) + (i - c_1) \big] \times L$$

2.5 稀疏矩阵

当矩阵中非零元素个数 $<5\%$ 时为稀疏矩阵。

- **压缩存储方式**：① 三元组表 `(i, j, value)`、② 十字链表法（结点含5个域：`row`, `col`, `val`, `down`, `right`）。
- **代价**：失去了随机存取功能。

第 3 章 栈和队列

3.1 栈 (Stack) —— 操作受限的线性表 (LIFO)

只允许在一端（栈顶 `top`）进行插入和删除。

> **主流约定（栈顶初始规约）**：
>
> - 若初始化 `top = 0`：入栈 `S[top++] = e`；出栈 `e = S[--top]`。
> - 若初始化 `top = -1`：入栈 `S[++top] = e`；出栈 `e = S[top--]`。*(本讲义采用第一种)*

顺序栈 C++ 经典类封装

C++

```
template <typename T>
class SeqStack {
private:
    T* data;
    int top;
    int max_size;
```

```
public:
    SeqStack(int size = 100) : max_size(size), top(0) {
        data = new T[max_size];
    }

    bool push(const T& e) {
        if (top >= max_size) return false; // 上溢
        data[top++] = e;
        return true;
    }

    bool pop(T& e) {
        if (top == 0) return false; // 下溢
        e = data[--top];
        return true;
    }

    ~SeqStack() { delete[] data; }
};
```

- **堆栈核心工程用途**：① 函数调用底层的栈帧维护；② **递归运算**；③ 保护现场与恢复现场。

3.2 循环队列 (Circular Queue) —— FIFO

核心痛点：普通顺序队列随着不断入队出队，指针只增不减，会导致“假溢出”。

解决途径：把一维数组在逻辑上卷成一个环。

循环队列四大判定公式（少用一个元素空间法）

设队列最大容量为 `MAXSIZE`：

1. **队空条件**：`front == rear`
2. **队满条件**：`(rear + 1) % MAXSIZE == front`
3. **队列元素个数**：

   $$L = (rear - front + MAXSIZE) \% MAXSIZE$$
4. **指针推演**：入队 `rear = (rear + 1) % MAXSIZE`；出队 `front = (front + 1) % MAXSIZE`。

C++

```
template <typename T>
class CircularQueue {
private:
    T* base;
    int front;
    int rear;
    int max_size;
public:
    CircularQueue(int size = 100) : max_size(size), front(0), rear(0) {
        base = new T[max_size];
    }

    bool enQueue(const T& e) {
        if ((rear + 1) % max_size == front) return false; // 队满
```

```
        base[rear] = e;
        rear = (rear + 1) % max_size;
        return true;
    }

    bool deQueue(T& e) {
        if (front == rear) return false; // 队空
        e = base[front];
        front = (front + 1) % max_size;
        return true;
    }
};
```

第 4 章 串 (String)

- **概念辨析**：
  - **空串 (Null String)**：长度为 0 的串；
  - **空白串 (Blank String)**：包含一个或多个**空格**的串。
- **核心定理**：空串是任意串的子串；任意串是其自身的子串；除本身外的子串称为**真子串**。

4.1 BF 模式匹配算法 (Brute-Force)

C++

```
#include <string>

// 标准 0-based 下标实现
int Index_BF(const std::string& S, const std::string& T, int pos = 0) {
    int i = pos;
    int j = 0;
    int s_len = S.length(), t_len = T.length();

    while (i < s_len && j < t_len) {
        if (S[i] == T[j]) {
            ++i; ++j;
        } else {
            i = i - j + 1; // 主串指针回溯到上一轮起点的后一个位置
            j = 0;         // 模式串归零
        }
    }
    return (j == t_len) ? (i - t_len) : -1;
}
```

> ⚠️ **考点批注**：原文档中的 `i = i - j + 2` 是严蔚敏版教材 `S[0]` 存放串长的 1-based 下标专属公式，期末写代码务必按照卷面给出的串定义选择回溯公式！

第 6 章 树与二叉树

6.1 二叉树五大必考性质

1. 在二叉树的第 i 层上至多有 **2^{i-1}** 个结点 ($i \ge 1$)。
2. 深度为 k 的二叉树至多有 **$2^k - 1$** 个结点 ($k \ge 1$)。
3. **叶子结点数公式（必考证明与填空）**：对任何一棵二叉树，若度为 2 的结点数为 n_2，则叶子数 n_0 公式为：

   $$n_0 = n_2 + 1$$

   (证明推导：总结点 $n = n_0+n_1+n_2$；总边数 $B = n-1 = n_1+2n_2$，两式联立相减即得)
4. 具有 n 个结点的**完全二叉树**的深度为 $\lfloor \log_2 n \rfloor + 1$。
5. **完全二叉树编号性质**（从上至下、从左至右，编号自 1 起）：
   - 左孩子编号：$2i$
   - 右孩子编号：$2i+1$
   - 双亲结点编号：$\lfloor i / 2 \rfloor$

6.2 二叉树遍历与森林转换

- **遍历次序**：
  - 先序遍历 (DLR)：根 $\rightarrow$ 左 $\rightarrow$ 右
  - 中序遍历 (LDR)：左 $\rightarrow$ 根 $\rightarrow$ 右
  - 后序遍历 (LRD)：左 $\rightarrow$ 右 $\rightarrow$ 根
- **唯一还原二叉树法则**：**（先序 或 后序）+ 中序** $\Rightarrow$ 可以唯一确定一棵二叉树。*仅凭先序和后序无法确定！*

树/森林 $\longleftrightarrow$ 二叉树转换法则（左孩子右兄弟）

- **连线**：将所有兄弟结点连接起来。
- **抹线**：对树中的每个结点，只保留它与长子（左孩子）的连线，删去与其他子女的连线。
- **旋转**：顺时针旋转 45°。

> *注：树转换成的二叉树，其**根结点必定没有右孩子**。*

6.3 哈夫曼树 (Huffman Tree)

- **定义**：带权路径长度 (WPL) 最短的二叉树（最优二叉树）。
- **构造原则**：**权值大的越靠近根，权值小的越远离根。**
- **带权路径公式**：

  $$WPL = \sum_{k=1}^{n} w_k l_k$$
- **哈夫曼编码**：前缀不等长编码，左分支标 `0`，右分支标 `1`，实现通信全路径最小冗余。

第 7 章 图 (Graph)

$$G = (V, E)$$

7.1 图的存储结构对比

1. **邻接矩阵 (Adjacency Matrix)**：
   - 用二维数组 `A[n][n]` 存边。
   - 空间复杂度 $O(n^2)$，**适合稠密图。**
   - 有向图中：第 i **行**非零元个数 = 顶点 v_i 的**出度**；第 i **列**非零元个数 = **入度。**

2. **邻接表 (Adjacency List)**：
    - 顺序存顶点头结点，每个头结点外挂一个邻接接边的单链表。
    - 空间复杂度 $O(n+e)$，**适合稀疏图。**

7.2 图的核心工程计算大题（期末重点）

问题一：最小生成树 (MST, Minimum Spanning Tree)

极小连通子图，含全部 n 个顶点，但**仅有 $n-1$ 条边。**

算法	核心核心策略	归并对象	时间复杂度	适用图类型
Prim（普里姆）	“步步为营”，不断找离当前生成树最近的**点**	**归并顶点**	$O(n^2)$	**稠密图**
Kruskal（克鲁斯卡尔）	“全局选优”，挑最小权值且不构成回路的**边**	**归并边**	$O(e \log_2 e)$	**稀疏图**

问题二：最短路径

1. **单源最短路径**：**Dijkstra（迪杰斯特拉）算法**（贪心思想，按路径长度递增次序产生，**边权不能为负**）。
2. **所有顶点间最短路径**：**Floyd（弗洛伊德）算法**（动态规划，三重循环 $O(n^3)$）。

第 8 章 查找 (Searching)

- **核心评估指标**：平均查找长度 $ASL = \sum P_i C_i$。

8.1 折半查找 (Binary Search)

- **前提**：① 顺序存储结构、② 记录按关键字有序。

C++

```
int BinarySearch(const int arr[], int n, int key) {
    int low = 0, high = n - 1;

    while (low <= high) {
        // 工程加分项：写法防止 low+high 整型溢出
        int mid = low + (high - low) / 2;

        if (arr[mid] == key) return mid;
        else if (arr[mid] > key) high = mid - 1;
        else low = mid + 1;
    }
    return -1;
}
```

- **折半查找判定树深度**：$\lfloor \log_2 n \rfloor + 1$。
- **成功时的 ASL 近似公式**：

  $$ASL \approx \log_2(n+1) - 1$$

8.2 二叉排序树 (BST) 与 平衡二叉树 (AVL)

- **二叉排序树性质**：左子树全小，右子树全大。**对其进行中序遍历，绝对得到一个递增有序序列。**
- **AVL 树定义**：任一结点的左、右子树深度之差的绝对值 ≤ 1。
  - **平衡因子 (BF)**：$BF = Depth{*left*} - *Depth*{right} \in {-1, 0, 1}$。
- **AVL 四大失衡调整旋转（手绘考点）**：
  1. **LL 型**：在左孩子的左子树插入 $\rightarrow$ **单右旋**（以 B 为轴）。
  2. **RR 型**：在右孩子的右子树插入 $\rightarrow$ **单左旋**（以 B 为轴）。
  3. **LR 型**：在左孩子的右子树插入 $\rightarrow$ **先左旋再右旋。**
  4. **RL 型**：在右孩子的左子树插入 $\rightarrow$ **先右旋再左旋。**

第 9 章 内部排序 (Sorting)

9.1 终极考点大纲横向对比大表

排序分类	算法名称	平均时间	最坏时间	辅助空间	稳定性	核心逻辑特征
插入类	直接插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	**稳定**	边插边有序，发牌逻辑
	折半插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	**稳定**	查得快，$O(n \log_2 n)$比较，移得多
	希尔排序 (Shell)	$O(n^{1.3})$	$O(n^2)$	$O(1)$	**不稳定**	缩小增量分组跳跃插入
交换类	冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	**稳定**	每趟挤出一个最大泡到最后

排序分类	算法名称	平均时间	最坏时间	辅助空间	稳定性	核心逻辑特征
	快速排序	**$O(n \log_2 n)$**	$O(n^2)$	$O(\log_2 n)$	**不稳定**	找枢轴 Partition 分治
选择类	简单选择排序	$O(n^2)$	$O(n^2)$	$O(1)$	**不稳定**	挑出剩余最小者与头互换
	堆排序 (Heap)	**$O(n \log_2 n)$**	**$O(n \log_2 n)$**	**$O(1)$**	**不稳定**	构造成完全二叉树选择
归并类	二路归并排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n)$	**稳定**	分治合流，空间换时间

> 🔥 **考点勘误大警报**：
>
> 原大纲未点透**简单选择排序的稳定性**。简单选择排序是**不稳定**的！
>
> *反例证明*：序列 `[5, 5*, 2]`，第一趟找最小元 `2` 与首位 `5` 交换，得到 `[2, 5*, 5]`，两个 `5` 的相对位置瞬间逆转。

9.2 快速排序核心分治核心代码 (Partition)

C++

```
int partition(int arr[], int low, int high) {
    int pivot = arr[low]; // 拿首元素作为枢轴中心

    while (low < high) {
        while (low < high && arr[high] >= pivot) --high;
        arr[low] = arr[high]; // 比枢轴小的甩到左边

        while (low < high && arr[low] <= pivot) ++low;
        arr[high] = arr[low]; // 比枢轴大的甩到右边
    }
    arr[low] = pivot; // 枢轴归位
    return low;
}

void QuickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot_pos = partition(arr, low, high);
        QuickSort(arr, low, pivot_pos - 1);
        QuickSort(arr, pivot_pos + 1, high);
    }
}
```